
系统开发工具基础

第 1 周实验报告

课程名称： 系统开发工具基础

授课教师： 周小伟

学生姓名： 洪朦朦

学 号： 24170001032

报告日期： 2026 年 9 月 2 日

GitHub: https://github.com/88594959h/Assignment_for_System_Development_Tools

1 实验概述

1.1 实验环境

- 操作系统: Windows 11 + Git Bash (MINGW64)
- Shell: Bash (Git for Windows)
- 版本控制: Git 2.x
- L^AT_EX 发行版: TeX Live 2024+
- 编辑器: VS Code

1.2 GitHub 仓库信息

- 仓库地址: https://github.com/88594959h/Assignment_for_System_Development_Tools

Commits on Aug 27, 2026	
q04 88594959h committed 4 days ago	7a96d0f <>
Commits on Aug 26, 2026	
q02: move analyze.sh into q02 88594959h committed 5 days ago	a35c7ef <>
q03: add terminal log 88594959h committed 5 days ago	d4f61ac <>
q03 88594959h committed 5 days ago	444a970 <>
q03 88594959h committed 5 days ago	48ccce4 <>
q02: auto-integrate script from history 88594959h committed 5 days ago	3ad1ccc <>
q02 88594959h committed 5 days ago	10f8f48 <>
Commits on Aug 25, 2026	
add solution script 88594959h committed last week	7f5b78a <>
q01 88594959h committed last week	66d3e2f <>

图 1: Git Commit 历史记录截图 (git log --oneline)

2 课上实验 (第 1 周)

2.1 实验一：含空格文件名的批量整理

2.1.1 题目要求

1. 创建 `input/docs` 和 `input/tmp`；创建含空格文件名的文件及隐藏文件。
2. 显示 `q01` 的绝对路径，以长格式列出 `input` 下全部项目（含隐藏文件）。
3. 将 `input` 下所有 `.txt` 文件复制到 `work/< 学号 >/`，保留相对目录结构并正确处理空格。
4. 设置目录权限 750、文件权限 640；生成 `inventory.txt`。

2.1.2 核心指令与实现

```
# 1. 创建目录结构
mkdir -p input/docs input/tmp work/24170001032

# 2. 创建文件（注意空格转义）
echo -e "alpha\nbeta" > "input/docs/notes one.txt"
echo "hidden" > input/docs/.secret.txt
touch input/tmp/empty.txt

# 3. 显示路径与列表
pwd
ls -la input/

# 4. 复制文件（处理空格的关键：find + cp --parents 或 引号）
# 方法：进入 input 目录操作以保持相对路径
(cd input && find . -name "*.txt" -exec cp --parents {} ../work
  /24170001032/ \;)

# 5. 设置权限
find work/24170001032 -type d -exec chmod 750 {} +
find work/24170001032 -type f -exec chmod 640 {} +

# 6. 生成 inventory.txt
(cd work/24170001032 && find . -type f -printf "%P %s\n" | sort >
  ../../inventory.txt)
```

Listing 1: 实验一关键 Shell 指令

2.1.3 实验分析与难点

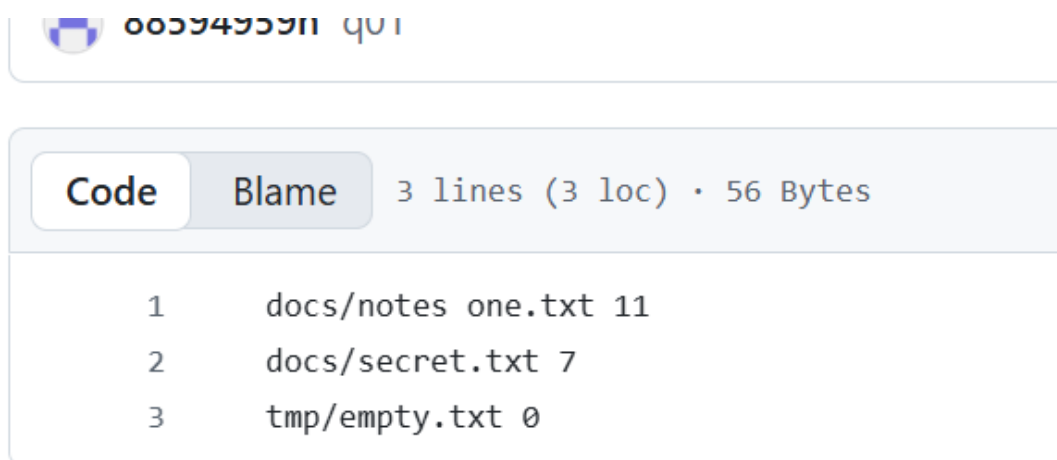
实现思路

利用 `find` 命令配合 `-exec` 或管道来处理包含空格的文件名，避免直接使用通配符 `*` 导致的单词拆分问题。`cp --parents` 选项用于保留源文件的相对目录结构。

实现重点

- **空格处理:** 在 Shell 中，文件名含空格必须用双引号包裹，或使用 `find -print0 | xargs -0` 范式。
- **权限理解:** 目录 750 (`rwxr-x—`) 允许所有者读写执行，组用户读执行；文件 640 (`rw-r—`) 允许所有者读写，组用户只读。

2.1.4 运行结果

图 2: 实验一: `inventory.txt` 内容截图

2.2 实验二：随机访问日志统计

2.2.1 题目要求

在 q02 中创建 `access.csv`，并使用下方给定数据完成日志统计。给定内容（`access.csv`）：

```
timestamp,user,path,status,latency_ms
09:00:01,alice,/api/users,200,120
09:00:02,bob,/api/orders,500,310
09:00:03,alice,/api/users,503,180
09:00:04,carol,/login,500,90
09:00:05,bob,/api/orders,502,260
09:00:06,dave,/health,200,20
09:00:07,alice,/api/orders,500,420
09:00:08,carol,/api/users,500,150
```

1. 编写 `analyze.sh`，接收一个 CSV 文件路径；文件不存在时将错误写入 `stderr` 并返回非零退出码。
2. 使用 Shell 管道以及 `awk`、`sort`、`head` 等工具，输出 HTTP 5xx 数量最多的前 2 个 path；按次数降序，次数相同时按 path 字典序。
3. 输出全部数据行的平均 `latency_ms`，保留两位小数；表头不得计入。
4. 分别对 `access.csv` 和一个不存在的文件运行脚本；不得使用 Python、电子表格或手工统计。

2.2.2 脚本实现 (analyze.sh)

```
#!/bin/bash
FILE=$1

# 检查文件是否存在
if [ ! -f "$FILE" ]; then
    echo "Error: File '$FILE' not found." >&2
    exit 1
fi

echo "=== Top 2 Paths with 5xx Errors ==="
```

```
# 跳过表头, 筛选 5xx 状态码 (第4列), 提取 path (第3列), 排序统计
tail -n +2 "$FILE" | awk -F',' ' $4 ~ /^5/ {print $3}' | sort | uniq
    -c | sort -rn -k1,1 -k2,2 | head -n 2

echo ""
echo "=== Average Latency ==="
# 计算平均 latency (第5列), 保留两位小数
tail -n +2 "$FILE" | awk -F',' '{sum+=$5; count++} END {if(count>0)
    printf "%.2f\n", sum/count; else print "0.00"}'
```

Listing 2: analyze.sh 脚本内容

2.2.3 运行结果

```
$ ./analyze.sh access.csv
=== Top 2 Paths with 5xx Errors ===
    2 /api/orders
    1 /api/users

=== Average Latency ===
193.75
```

Listing 3: 正常文件运行结果

```
$ ./analyze.sh nonexistent.csv
Error: File 'nonexistent.csv' not found.
$ echo $?
1
```

Listing 4: 异常文件运行结果

2.3 实验三：制造、解决并解释一次合并冲突

2.3.1 题目要求

在 q03 中从零创建一个 Git 仓库，并主动制造、解决一次合并冲突。

1. 初始化仓库，在 main 创建 config.txt，内容为 mode=normal，并完成初始提交。
2. 创建 feature-a，把内容改为 mode=safe 并提交；从初始 main 创建 feature-b，把内容改为 mode=fast 并提交。
3. 回到 main，先合并 feature-a，再合并 feature-b；解决冲突时保留 mode=safe，并另加一行 note=reviewed。
4. 确认工作区干净，运行 `git log --all --graph --decorate --oneline` 查看历史。

2.3.2 Git 操作流程

```
# 1. 初始化与初始提交
git init -b main
echo "mode=normal" > config.txt
git add config.txt
git commit -m "Initial commit with mode=normal"

# 2. Feature A 分支
git checkout -b feature-a
echo "mode=safe" > config.txt
git add config.txt
git commit -m "Change mode to safe in feature-a"

# 3. Feature B 分支 (从 main 分出)
git checkout main
git checkout -b feature-b
echo "mode=fast" > config.txt
git add config.txt
git commit -m "Change mode to fast in feature-b"

# 4. 合并与冲突解决
git checkout main
git merge feature-a # Fast-forward 成功
```

```
git merge feature-b # 产生 CONFLICT

# 5. 手动解决冲突
echo "mode=safe" > config.txt
echo "note=reviewed" >> config.txt
git add config.txt
git commit -m "Merge feature-b: resolve conflict, keep safe mode"

# 6. 查看历史
git log --all --graph --decorate --oneline
```

Listing 5: Git 操作指令序列

2.3.3 冲突解决分析

原理说明

当两个分支修改了同一文件的同一行时，Git 无法自动合并，会标记冲突。此时文件中会出现 <<<<<<, =====, >>>>>> 标记。我们需要手动编辑文件保留 desired content，然后 git add 标记为已解决，最后 git commit 完成合并。

2.3.4 运行结果

```
* a616d51 (HEAD -> main) Merge feature-b: resolve conflict...

|\
| * 3f9b85f (feature-b) Change mode to fast in feature-b
* | 71f7c45 (feature-a) Change mode to safe in feature-a

|/
* 1d8c934 Initial commit with mode=normal
```

Listing 6: Git Log 图形化输出

2.4 实验四：修复并构建一页技术说明 (LaTeX)

2.4.1 题目要求

补全 LaTeX 模板，加入公式、表格及交叉引用，使用 `latexmk` 生成 PDF。

2.4.2 实现代码片段

```
\section{Result}
爱因斯坦质能方程如公式 \eqref{eq:einstein} 所示：
\begin{equation}
    E = mc^2
    \label{eq:einstein}
\end{equation}

实验数据汇总见表 \ref{tab:data}。
\begin{table}[H]
    \centering
    \caption{实验数据记录}
    \label{tab:data}
    \begin{tabular}{cc}
        \toprule
        \textbf{变量} & \textbf{值} \\
        \midrule
        Mass & 1.0 kg \\
        Energy & 9.0e16 J \\
        \bottomrule
    \end{tabular}
\end{table}
```

Listing 7: report.tex 关键片段

2.4.3 编译指令

```
latexmk -pdf -halt-on-error report.tex
```

Listing 8: 编译命令

2.4.4 运行结果



图 3: 实验四：生成的 PDF 页面截图

3 课后思考题

3.1 思考题 1：标准流重定向

题目：运行 `ls /nonexistent /tmp`，把 `stdout` 和 `stderr` 分别重定向到两个文件。如何将两者都重定向到同一个文件？

解答：

```
# 分别重定向
ls /nonexistent /tmp > out.txt 2> err.txt

# 合并重定向 (2>&1 表示将 stderr 指向 stdout 当前指向的地方)
ls /nonexistent /tmp > all.txt 2>&1
```

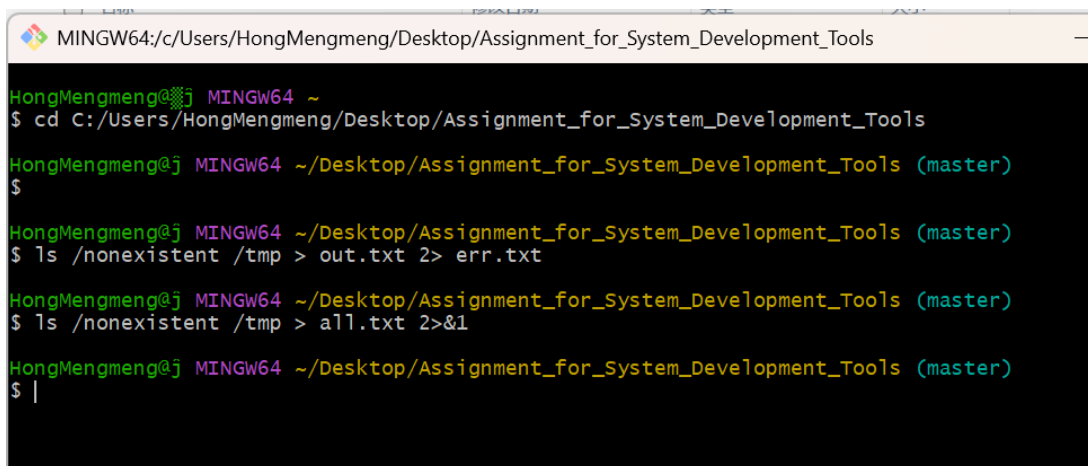


图 4：两种情况示例

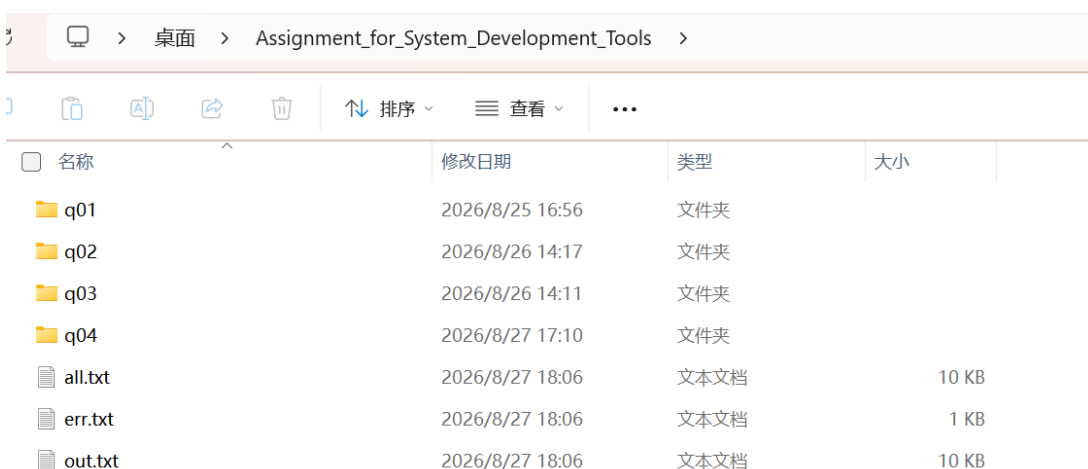


图 5：变化情况 1

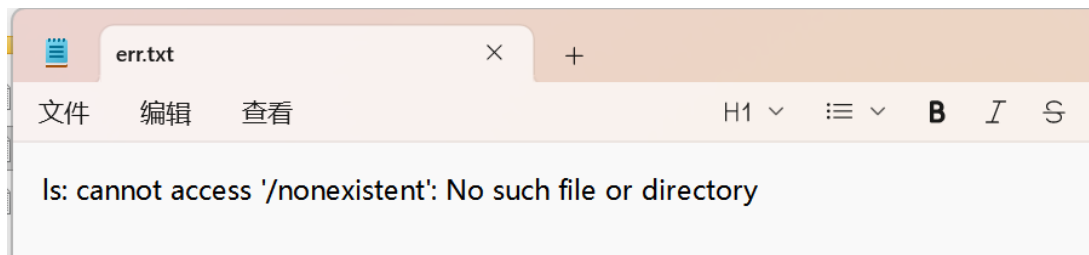


图 6: 变化情况 2

3.2 思考题 2: 条件执行与目录创建

题目: 写一个一行命令: 仅当 /tmp/mydir 不存在时才创建它。

解答: 利用 || (OR) 操作符, 前一条命令失败 (即目录不存在, 测试返回非 0) 时执行后一条。

```
[ -d /tmp/mydir ] || mkdir /tmp/mydir
```

或者使用更简洁的 `mkdir -p` (虽不完全是题目要求的逻辑, 但效果类似且幂等)。



图 7: 第一次及第二次运行

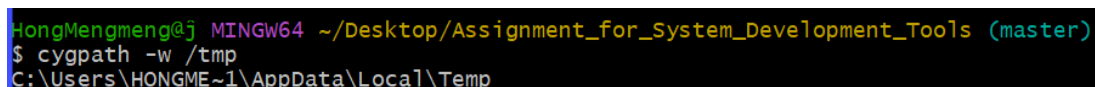


图 8: 绝对路径

3.3 思考题 3: cd 为什么必须是内建命令

题目: 为什么 cd 必须是 Shell 内建命令, 而不能是独立程序?

解答:

核心原因

子进程不能改变父进程的工作目录。Shell 执行外部命令时会 fork 一个子进程。子进程结束后，其对环境（包括当前工作目录 CWD）的任何修改都会随之消失，无法回传给父进程（Shell）。如果 cd 是外部程序，它只能改变自己子进程的目录，Shell 本身的目录不会变，命令也就失去了意义。因此，cd 必须由 Shell 自己在内部执行。

```
HongMengmeng@j MINGW64 ~  
$ type cd  
cd is a shell builtin  
  
HongMengmeng@j MINGW64 ~  
$ type ls  
ls is aliased to `ls -F --color=auto --show-control-chars`
```

图 9: 运行示例

3.4 思考题 4: 文件存在性检查脚本

题目：写一个脚本，接收文件名参数，检查是否存在并输出提示。

解答：

```
#!/bin/bash  
if [ -f "$1" ]; then  
    echo "File '$1' exists."  
else  
    echo "File '$1' does not exist."  
fi
```

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools (master)  
$ cat > check.sh << 'EOF'  
#!/bin/bash  
if [ -f "$1" ]; then  
    echo "exit"  
else  
    echo "NO exit"  
fi  
EOF  
  
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools (master)  
$ bash check.sh check.sh  
exit  
  
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools (master)  
$  
bash check.sh 不存在的文件.xyz  
NO exit
```

图 10: 运行示例

3.5 思考题 5: set -x 的作用

题目：在脚本的 set 选项里加入 -x 会发生什么？

解答：set -x 开启 执行追踪 (Execution Tracing) 模式。Shell 在执行每条命令前，会将其（展开变量后）打印到 stderr，并以 + 开头。这是调试脚本最有效的手段，可以清晰看到变量的真实值和命令的执行流程。

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools (master)
$ cat > demo.sh << 'EOF'
#!/bin/bash
name="world"
echo "Hello, $name"

for i in 1 2 3; do
    echo "count = $i"
done

result=$(date +%Y-%m-%d)
echo "Today is $result"
EOF
```

图 11: 创建一个简单的脚本文件

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools (master)
$ bash demo.sh
Hello, world
count = 1
count = 2
count = 3
Today is 2026-08-28
```

图 12: 普通输出

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools (master)
$ bash -x demo.sh
+ name=world
+ echo 'Hello, world'
Hello, world
+ for i in 1 2 3
+ echo 'count = 1'
count = 1
+ for i in 1 2 3
+ echo 'count = 2'
count = 2
+ for i in 1 2 3
+ echo 'count = 3'
count = 3
++ date +%Y-%m-%d
+ result=2026-08-28
+ echo 'Today is 2026-08-28'
Today is 2026-08-28
```

图 13: 加上-x 输出

3.6 思考题 6: 带日期的文件备份

题目：写一条命令，把文件复制为带当天日期的备份文件名。

解答：利用命令替换 \$(...) 获取日期。

```
cp notes.txt notes_$(date +%Y-%m-%d).txt
```

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/test (master)
$ echo "This is a note">notes.txt

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/test (master)
$ cp notes.txt notes_$(date +%Y-%m-%d).txt

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/test (master)
$ ls -l
total 2
-rw-r--r-- 1 HongMengmeng 197609 15 Aug 28 14:01 notes.txt
-rw-r--r-- 1 HongMengmeng 197609 15 Aug 28 14:01 notes_2026-08-28.txt
```

图 14: 运行示例

3.7 思考题 7: 统计常见文件扩展名

题目: 使用管道找出 home 目录中最常见的 5 种文件扩展名。

解答:

```
find ~ -type f | grep -oE '\.[a-zA-Z0-9]+$' | sort | uniq -c | sort
-rn | head -5
```

解析: find 找文件 → grep 提取扩展名 → sort 排序 → uniq -c 计数 → sort -rn 降序 → head 取前 5。

```
-rw-r--r-- 1 HongMengmeng 197609 15 Aug 28 14:01 notes_2026-08-28.txt
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/test (master)
$ find ~ -type f | grep -oE '\.[a-zA-Z0-9]+$' | sort | uniq -c | sort -rn | head -5
Find: '/c/Users/HongMengmeng/AppData/Local/Lenovo/devicecenter/cefcache/widevinecdm/4.10.2891.0': Permission denied
Find: '/c/Users/HongMengmeng/AppData/Local/Lenovo/LeAppStore/MsgCache/pool_0/widevinecdm/4.10.2891.0': Permission denied
Find: '/c/Users/HongMengmeng/AppData/Local/Lenovo/LeAppStore/MsgCache/pool_1/widevinecdm/4.10.2891.0': Permission denied
Find: '/c/Users/HongMengmeng/AppData/Local/Lenovo/LeAppStore/storecache/widevinecdm/4.10.2891.0': Permission denied
Find: '/c/Users/HongMengmeng/AppData/Local/Microsoft/windows/INetCache/Low/Content.IE5': Permission denied
Find: '/c/Users/HongMengmeng/AppData/Local/Qianwen/User Data/AutoFillStates/2025.6.13.84507': Permission denied
Find: '/c/Users/HongMengmeng/AppData/Local/Qianwen/User Data/CertificateRevocation/10628': Permission denied
Find: '/c/Users/HongMengmeng/AppData/Local/Qianwen/User Data/Crowd Deny/2026.6.29.61': Permission denied
Find: '/c/Users/HongMengmeng/AppData/Local/Qianwen/User Data/FileTypePolicies/145.0.7584.0': Permission denied
Find: '/c/Users/HongMengmeng/AppData/Local/Qianwen/User Data/FirstPartySetsPreloaded/2025.7.24.0': Permission denied
Find: '/c/Users/HongMengmeng/AppData/Local/Qianwen/User Data/hyphen-data/120.0.6050.0': Permission denied
Find: '/c/Users/HongMengmeng/AppData/Local/Qianwen/User Data/IEP/Preload/1.1.0.3': Permission denied
Find: '/c/Users/HongMengmeng/AppData/Local/Qianwen/User Data/OptimizationHints/703': Permission denied
Find: '/c/Users/HongMengmeng/AppData/Local/Qianwen/User Data/PKIMetadata/1707': Permission denied
Find: '/c/Users/HongMengmeng/AppData/Local/Qianwen/User Data/PrivacySandboxAttestationsPreloaded/2025.6.16.0': Permission denied
Find: '/c/Users/HongMengmeng/AppData/Local/Qianwen/User Data/SafetyTips/3091': Permission denied
Find: '/c/Users/HongMengmeng/AppData/Local/Qianwen/User Data/SSLErrorAssistant/7': Permission denied
Find: '/c/Users/HongMengmeng/AppData/Local/Qianwen/User Data/Subresource Filter/Unindexed Rules/9.69.0': Permission denied
Find: '/c/Users/HongMengmeng/AppData/Local/Qianwen/User Data/TrustTokenKeyCommitments/2026.3.23.1': Permission denied
Find: '/c/Users/HongMengmeng/AppData/Local/Qianwen/User Data/ZxcvbnData/3': Permission denied
Find: '/c/Users/HongMengmeng/AppData/Local/Temp/devicecenter/cache/88DE0184-F012-4F38-9FDC-57780C465EF2/widevinecdm/4.10.2891.0': Permission denied
Find: '/c/Users/HongMengmeng/AppData/Local/Temp/LTSF_B0FA6.tmp': Permission denied
32797 .svg
14942 .xz
10255 .js
8539 .png
7759 .d11
```

图 15: 运行示例

3.8 思考题 8: xargs 与空格处理

题目: 结合 find 和 xargs 统计 .sh 文件行数, 正确处理空格。

解答: 使用 -print0 和 -0 配对, 以 NULL 字符分隔, 彻底避免空格/换行符干扰。

```
find ~ -type f -name '*.sh' -print0 | xargs -0 wc -l
```

```
ass.sh
6 /c/Users/HongMengmeng/Desktop/Assignment_for_System_Development_Tools/check.sh
10 /c/Users/HongMengmeng/Desktop/Assignment_for_System_Development_Tools/demo.sh
15 /c/Users/HongMengmeng/Desktop/Assignment_for_System_Development_Tools/q01/solution.sh
4 /c/Users/HongMengmeng/Desktop/Assignment_for_System_Development_Tools/q02/analyze.sh
191 /c/Users/HongMengmeng/Desktop/Assignment_for_System_Development_Tools/q02/run_all.sh
85 /c/Users/HongMengmeng/Desktop/class_work/multi_cycle_cpu/multi_cycle_cpu.runs/impl_1/ISEWrap.sh
48 /c/Users/HongMengmeng/Desktop/class_work/multi_cycle_cpu/multi_cycle_cpu.runs/impl_1/runme.sh
85 /c/Users/HongMengmeng/Desktop/class_work/multi_cycle_cpu/multi_cycle_cpu.runs/synth_1/ISEWrap.sh
44 /c/Users/HongMengmeng/Desktop/class_work/multi_cycle_cpu/multi_cycle_cpu.runs/synth_1/runme.sh
85 /c/Users/HongMengmeng/Desktop/class_work/project_3/project_3.runs/synth_1/ISEWrap.sh
44 /c/Users/HongMengmeng/Desktop/class_work/project_3/project_3.runs/synth_1/runme.sh
14361 total
```

图 16: 统计结果

3.9 思考题 9: Shell 引用机制

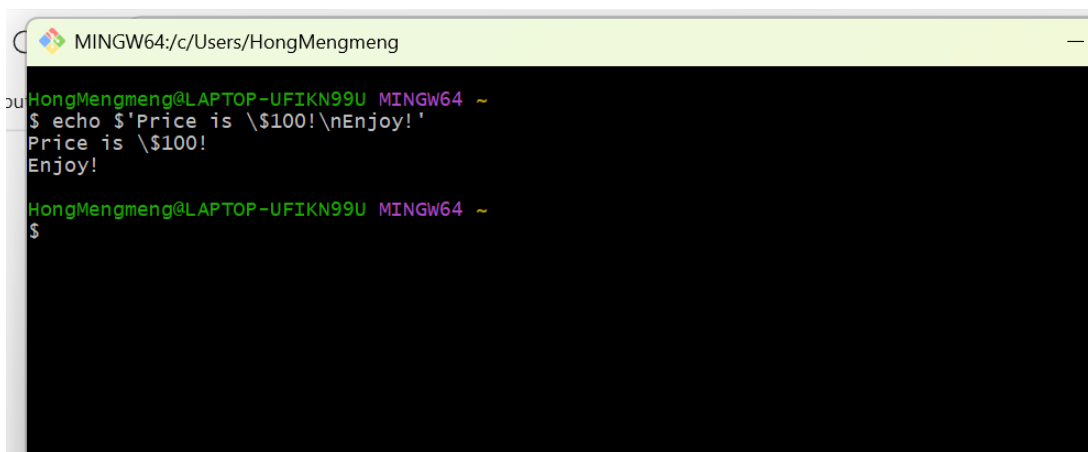
题目: 单引号、双引号和 ANSI-C 引号的区别? 输出含 \$, !, 换行符的字符串。

解答:

- 单引号 '...': 强引用, 内容完全原样输出, 不解析任何变量或转义。
- 双引号 "...": 弱引用, 解析变量 \$var 和命令替换 \$(cmd), 但保留空格。
- ANSI-C '\$'...': 支持 C 风格转义, 如 \n (换行), \t (制表符)。

命令:

```
echo '$Price is \ $100!\nNew Line here.'
```



The screenshot shows a terminal window titled 'MINGW64/c/Users/HongMengmeng'. The prompt is 'HongMengmeng@LAPTOP-UFIKN99U MINGW64 ~'. The command entered is '\$ echo '\$Price is \ \$100!\nEnjoy!'''. The output is 'Price is \ \$100!' followed by a new line and 'Enjoy!'.

图 17: 运行示例

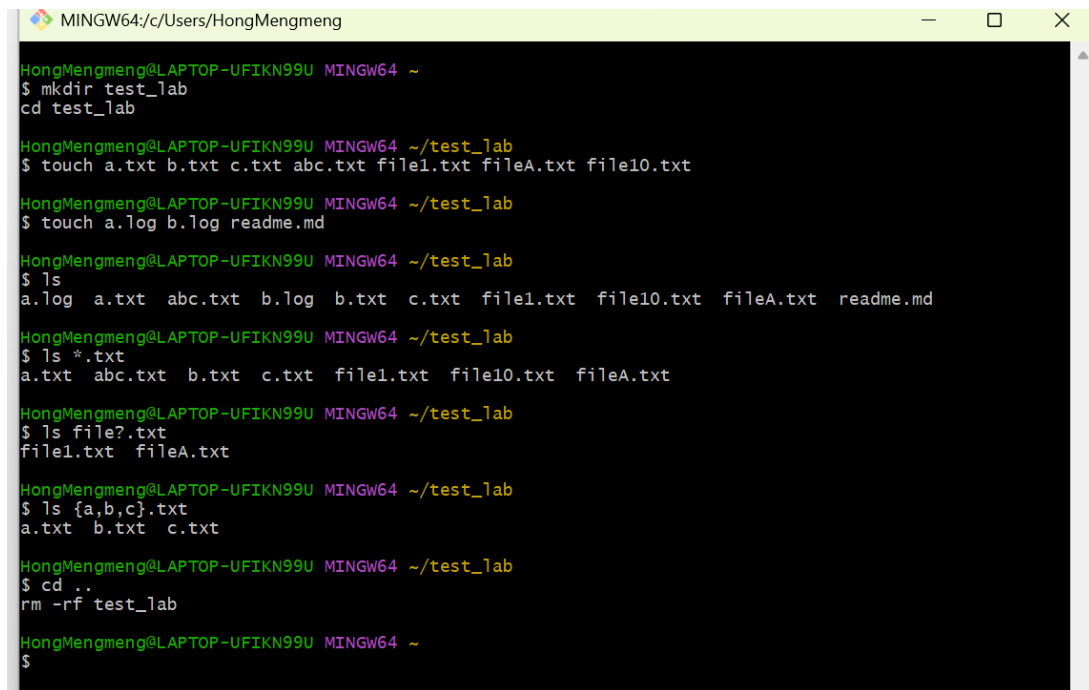
3.10 思考题 10: Glob 模式匹配

题目: 什么是 glob? 测试 ls *.txt 等模式。

解答: Glob 是 Shell 内置的文件名通配符匹配机制, 由 Shell 在命令执行前展开为文件列表。

- *: 匹配任意长度字符串（不含 /）。
- ?: 匹配单个字符。
- [abc]: 匹配括号内任一字符。
- {a,b}: 花括号展开（Brace Expansion），生成多个参数。

注意：Glob 不是正则表达式，它只用于匹配文件名。



```
MINGW64: c:/Users/HongMengmeng
HongMengmeng@LAPTOP-UFIKN99U MINGW64 ~
$ mkdir test_lab
$ cd test_lab

HongMengmeng@LAPTOP-UFIKN99U MINGW64 ~/test_lab
$ touch a.txt b.txt c.txt abc.txt file1.txt fileA.txt file10.txt

HongMengmeng@LAPTOP-UFIKN99U MINGW64 ~/test_lab
$ touch a.log b.log readme.md

HongMengmeng@LAPTOP-UFIKN99U MINGW64 ~/test_lab
$ ls
a.log a.txt abc.txt b.log b.txt c.txt file1.txt file10.txt fileA.txt readme.md

HongMengmeng@LAPTOP-UFIKN99U MINGW64 ~/test_lab
$ ls *.txt
a.txt abc.txt b.txt c.txt file1.txt file10.txt fileA.txt

HongMengmeng@LAPTOP-UFIKN99U MINGW64 ~/test_lab
$ ls file?.txt
file1.txt fileA.txt

HongMengmeng@LAPTOP-UFIKN99U MINGW64 ~/test_lab
$ ls {a,b,c}.txt
a.txt b.txt c.txt

HongMengmeng@LAPTOP-UFIKN99U MINGW64 ~/test_lab
$ cd ..
$ rm -rf test_lab

HongMengmeng@LAPTOP-UFIKN99U MINGW64 ~
$
```

图 18: 运行示例

3.11 思考题 11: ls -l 输出格式解析

题目: ls 的 -l 选项 (flag) 作用是什么? 运行 `ls -l /` 并观察输出。每一行最前面的 10 个字符分别代表什么?

解答:

ls -l 的作用

ls -l 以长格式 (long listing) 显示文件或目录的详细信息, 包括: 文件类型与权限、硬链接数、所有者、所属组、文件大小 (字节)、最后修改时间、文件名 (或目录名)。

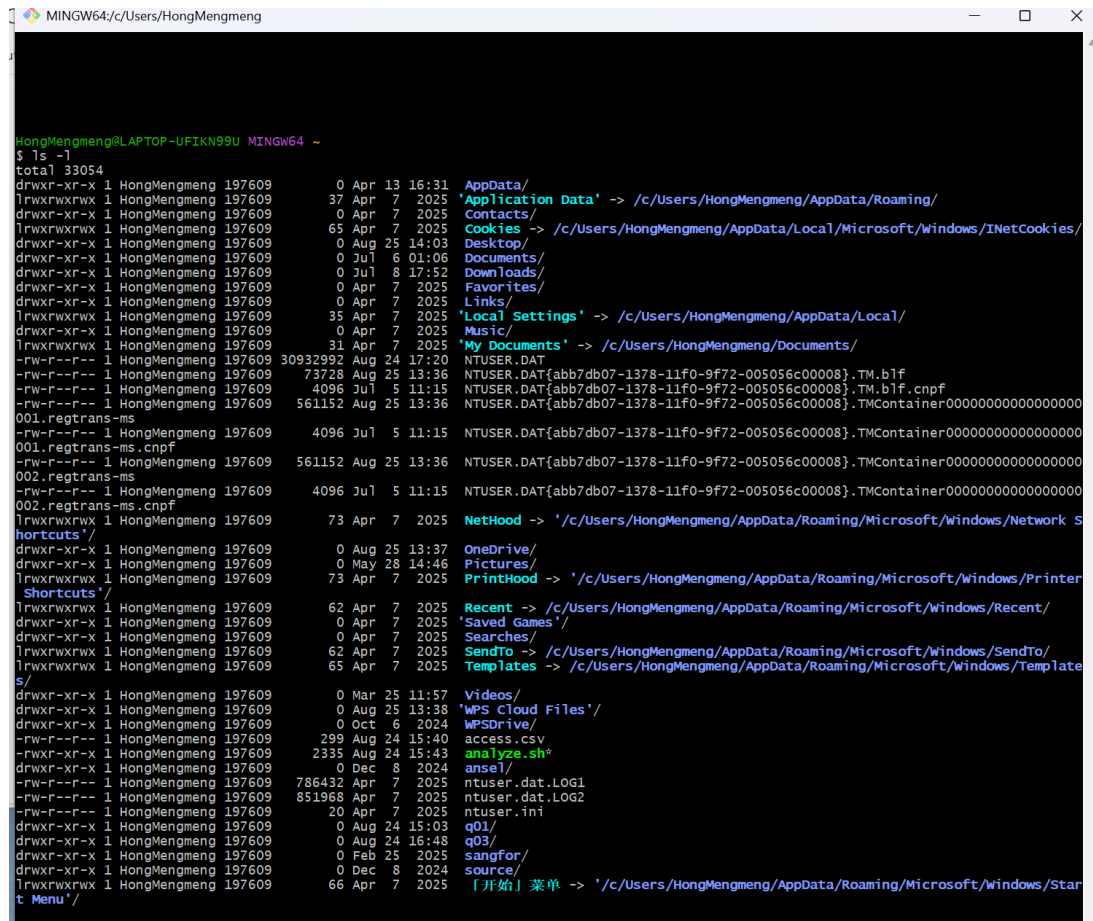


图 19: 输出示例

每行最前面 10 个字符的含义如下：第 1 位：文件类型

字符	含义
d	目录 (directory)
-	普通文件 (regular file)
l	符号链接 (symbolic link)
c	字符设备 (character device)
b	块设备 (block device)
p	命名管道 (named pipe / FIFO)
s	套接字 (socket)

第 2-10 位：权限位（3 组 × 3 位）

- 第 2-4 位：所有者（user）权限
- 第 5-7 位：所属组（group）权限
- 第 8-10 位：其他人（others）权限

每组三位依次为 **r**(读)、**w**(写)、**x**(执行); 对应位置为 **-** 表示无该权限。例如 **-rw-r--r--** 表示: 普通文件, 所有者可读写, 组用户和其他人仅可读。

4 实验总结与感悟

4.1 遇到的问题与解决方法

1. **文件名空格问题**：在实验一中，直接使用 `cp input/docs/*.txt` 会导致含空格的文件名被截断。通过改用 `find ... -exec` 或给变量加双引号 `"$file"` 解决。
2. **Git 冲突标记**：初次遇到冲突时不知所措，通过查阅文档理解了 `<<<<<<<` 标记的含义，手动编辑文件后记得 `git add` 才能标记为已解决。
3. **LaTeX 字体报错**：最初手动指定 `SimSun` 导致编译失败，改为依赖 `ctex` 自动检测系统字体后解决。

4.2 心得体会

通过本次实验，我深刻体会到了命令行工具的强大之处。特别是管道 `|` 的设计哲学——“每个程序只做一件事，并做好”，通过组合小工具完成复杂任务（如题 7 的统计），比编写庞大的单体程序更高效。同时，Git 的分支与合并机制让我理解了版本控制在协作中的核心价值。LaTeX 虽然上手有门槛，但其对学术排版的严谨性令人印象深刻。